

Introduction and Course Overview

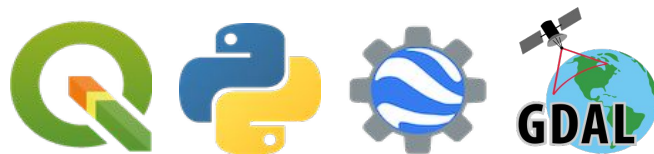
Agentic Coding for Geospatial

Ujaval Gandhi
Spatial Thoughts
ujaval@spatialthoughts.com



Spatial Thoughts

- A learning platform for modern geospatial technologies.
- Free and open learning content for QGIS, Python, GDAL and Google Earth Engine.
 - Learning materials used by 1 million+ users every year.
 - All material is licensed under CC-BY allowing anyone to use and adapt for free
- Online academy for cohort-based instructor-led classes.
 - Trained participants from over 100 countries.



www.spatialthoughts.com

Introduction

- **Background in GIS and Remote Sensing**
 - Intern at Indian Institute of Remote Sensing (IIRS), Dehradun, India
 - MS in Geospatial Information Engineering from University of Wisconsin - Madison, USA
- **15 years of professional experience**
 - One of the early employees at Google Inc.
 - Built large scale data processing pipelines for Google Earth
 - Led the team responsible for satellite and aerial imagery for Google Maps
 - Developed expertise in Google Earth Engine and trained 1000+ scientists and researchers across India
- **Presently**
 - Left Google in 2020 to work on startup Spatial Thoughts.
 - Building open-source content and helping people master advanced geospatial skills.



Ujaval Gandhi

Introduction

- Masters in Remote Sensing
- Interest in Automating Workflows
- Training Associate
 - Class logistics
 - Project support



Vigna Purohit

Course Overview

Module 1

Getting Started with Agentic Coding

Setting up your Agent

Application 1: Geospatial Data Analysis
(Mapping Crime Hotspots)

Module 2

Building Complex Workflows

Application 2: Using Web APIs
(Route Optimization)

Agent Skills

Course Overview

Module 3

AI Second Brain

Module 4

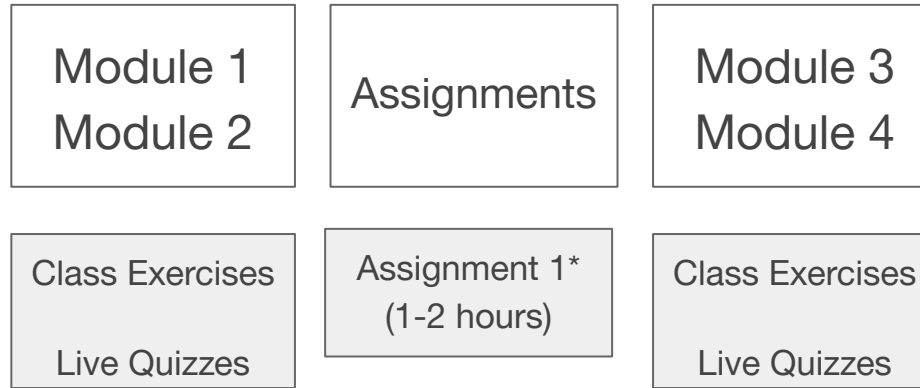
GeoAI

Application 3: Building a Personal
Knowledge Website

MCP Servers

Application 4: Object Detection
using Deep Learning

Assignment and Evaluation



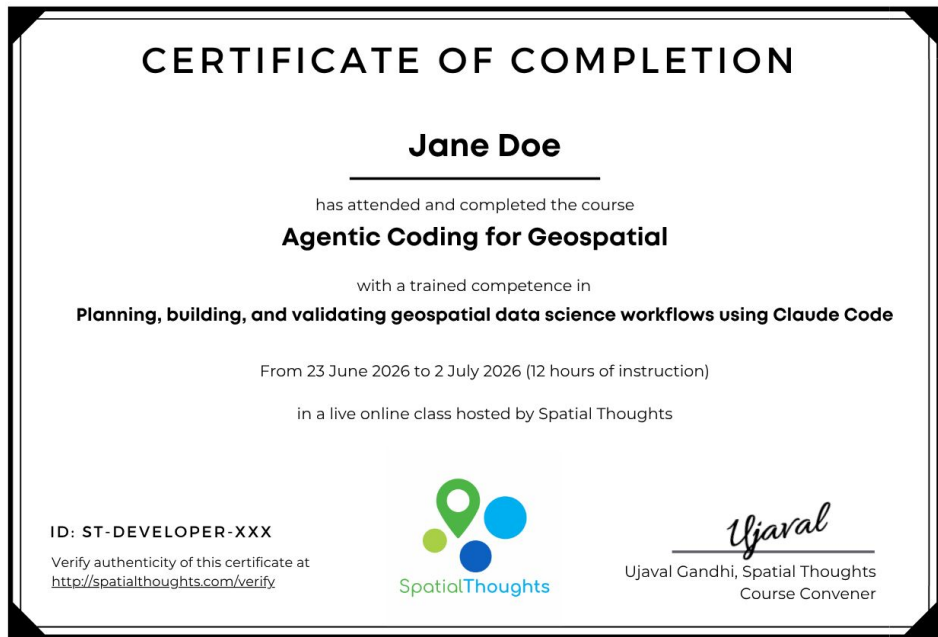
** Assignment is due on the last day of the course*

Live Session Structure

- All the live sessions are hands-on.
- We will have 10-15 minutes of explanation of a concept, followed by a 5-minute exercise
 - All participants are expected to complete the exercises during the class.
- You can ask questions at any point during the session. Just unmute and ask, or use the chat.
- We will have 1 break of 10 minutes during each 3-hour session.
- All sessions are recorded and will be shared with the participants. If you miss any session, watch the recording and submit the exercises by email.

Certification

- Each participant would be given a certificate of completion
- Certificates are issued by Spatial Thoughts and come with a unique ID that can be verified at spatialthoughts.com/verify
- Participants must be **enrolled in a course** and complete the exercises and assignment to earn the certificate.
- If you miss a class, you should submit the exercises by email for evaluation.





Introduction to Agentic Coding

AI Agents

- Agents are software systems that are able to complete tasks on behalf of the users.
- Instead of user prompting the model at each step for a response, Agents can plan, execute multi-step workflows.
- The core principle is the **ReAct** loop (reasoning and acting) [[reference](#)]
 - Reason → Act → Observe → Repeat

Agentic Coding

- Agentic coding involves setting high-level goals than line-by-line prompts or copy/pasting code from AI Assistants.
- The agent plans a coding task, executes changes across multiple files, runs the code, fixes errors, checks the outputs and iterates based on results.
 - If the code throws an error, the agent reads the output, reasons through the failure, and applies a fix on its own
- This paradigm shifts the developer's role from writing every line to reviewing, directing, and validating the agent's work.

Agent Harness

- LLMs themselves can only generate a response from a prompt.
- You need a system that can use LLMs to perform agentic codings tasks. This system is the “Agent Harness”.
 - Harness turns LLMs into Agents

Agent = Model + Harness

Key Features of a Agent Harness

- Tool Access
 - Has access to tools to manage code (i.e. Git), run shell commands, call APIs etc.
- Memory
 - Has context on the previous executions and user preferences
- Workspace
 - Has access to files, runtime environment and operating system
- Guardrails
 - Has the ability to manage permissions, and enforce policies

Claude Code

- Claude Code is the agentic harness around Claude models by Anthropic.
- One of the most popular coding agent in the market.
- It is a closed-source system that is tightly integrated with Claude models and Anthropic APIs.
- Other competitors to Claude Code include
 - [OpenAI Codex](#)
 - [Gemini CLI](#)
 - [GitHub Copilot CLI](#)
- The ecosystem is more-or-less standardized on the feature set.
 - Cost and Model performance are the key factors in determine your choice of agent harness.

Open Source Coding Agents

- Many open-source alternatives exist that provide similar functionality.
 - [OpenCode](#)
 - [Pi](#)
 - [DeepSeek Harness](#)
- Model choice and flexibility are the main factors driving adoption of open-source harnesses.
 - You can pick from a wide range of models
 - You can use local models running on your own machine
 - You get a lot more visibility and transparency on the execution flow.



Let's get coding

Get the Workshop Materials

Workshop materials are available at

<https://courses.spatialthoughts.com/agentic-coding-geospatial.html>



Introduction to Markdown

Markdown

- Markdown is a simple formatting language to create structured documents.
- Plain text formatting that is easy to read and write for humans and computers.
- It is a widely used standard for creating documents and websites
- Markdown documents can be easily converted to other formats (PDF, Word, HTML) without worrying about styling and format-specific formatting.

Markdown Text

```
# CLAUDE.md

## Geospatial Stack Preferences

- Prefer Python-based approaches
- Use existing packages instead of building solutions from scratch
- My preference for geospatial packages are below
  - Pandas for tabular data
  - GeoPandas for vector geospatial data
  - XArray ecosystem (rioxarray, xarray-spatial etc) for raster geospatial data
  - Scikit-learn for Machine Learning
  - GeoAI (geoai-py) for Deep Learning
- Web Apps
  - Streamlit for data driven apps
  - Leaflet for interactive mapping apps
  - Self contained HTML for small apps
```

Rendered HTML

CLAUDE.md

Geospatial Stack Preferences

- Prefer Python-based approaches
- Use existing packages instead of building solutions from scratch
- My preference for geospatial packages are below
 - Pandas for tabular data
 - GeoPandas for vector geospatial data
 - XArray ecosystem (rioxarray, xarray-spatial etc) for raster geospatial data
 - Scikit-learn for Machine Learning
 - GeoAI (geoai-py) for Deep Learning
- Web Apps
 - Streamlit for data driven apps
 - Leaflet for interactive mapping apps
 - Self contained HTML for small apps

Flavors of Markdown

- The original Markdown specifications supports basic syntax - which is limiting for many applications.
- There is a superset of Markdown that support an extended syntax.
 - Tables
 - Code Blocks
 - Footnotes
 - ...
- GitHub supports the GitHub-flavored Markdown syntax which supports the full extended syntax, plus some GitHub-specific additions.
 - Tip, Notes, Alerts
 - Diagrams, Mathematical expressions
 - Referencing GitHub issues, users

Markdown is the language of AI

- AI systems use Markdown as a native format to structure content
 - Markdown is easier for LLMs to parse and makes them more effective than HTML/PDFs
- Markdown in prompts helps AI recognize structure and prioritize instructions.
 - Headings, bullet points, tables convey useful structure information
- Agentic systems like Claude Code use Markdown as the language for configuration.

